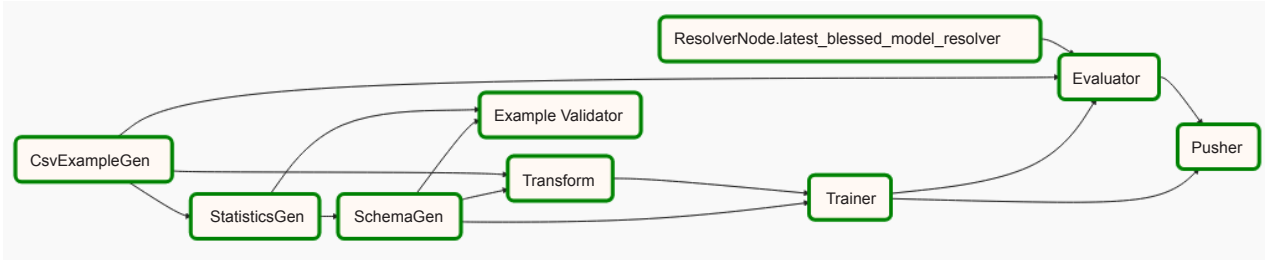

Figure 2: Chicago taxi pipeline DAG

use the following commands:

```
$export AIRFLOW_HOME=~/airflow
$export TFX_DIR=~/tfx
$pip install apache-airflow==1.10.12
--no-cache-dir
$pip uninstall cattrs==1.1.1
$pip install cattrs==1.0.0 --no-cache-
dir
$pip install tfx==0.24.1 --no-cache-dir
```

Code walkthrough

In this article we use one of the TFX pipeline examples given in the TFX library itself. The dataset consists of taxi rides taken in Chicago city in the US. The model built is used to predict whether a passenger will offer a tip to the taxi driver.

The TFX pipeline first takes the data in the form of CSV files and then converts it into the TensorFlow format. Next, various statistics such as the frequency distribution of categorical features and arithmetic means of numerical features are calculated. If no schema is given as input, then the schema of the data is also inferred. Otherwise, the data is validated against the schema given as input. It is then transformed to perform feature engineering, and given to the trainer which trains a ML model on the data. The model is then evaluated and if it passes the benchmarks set by us, it is given to the pusher. The pusher saves the model to a predefined folder. To see the pipeline as a DAG in Airflow, click on the workflow and then on *Graph View* on the Airflow dashboard. Figure 2 shows the model pipeline visualised in Apache Airflow.

There are also other viewing options in Airflow such as Tree View. The advantage of the workflow being a graph is that nodes (operations) that do not depend on each other can be scheduled and executed in parallel.

Once we have saved the model, it's time to serve it using the TensorFlow Serving API so that it can be used by other applications. We use shell script for automating this process. The script runs the model in a Docker container. It first pulls the *tensorflow-serving-api* image and then adds the model to the container. Thus, the container behaves as a model server, and we can send *gRPC* requests to get predictions from the model using a client script. Code snippets of the server and client scripts are shown in Figures 3 and 4, respectively.

```
docker stop chicago_container

docker rm chicago_container

docker run -d -p 127.0.0.1:$HOST_PORT:$CONTAINER_PORT \
  -v $LOCAL_MODEL_DIR:$CONTAINER_MODEL_DIR \
  -e MODEL_NAME=chicago_taxi \
  --name chicago_container \
  --rm $DOCKER IMAGE NAME
```

Figure 3: Script to start the model server

```
python `dirname "$(readlink -f "$0")"`/chicago_taxi_client.py \
  --num_examples 3 \
  --examples_file ${EXAMPLES_FILE} \
  --schema_file ${SCHEMA_FILE} \
  --server 127.0.0.1:9000
```

Figure 4: Script to run inference requests on the model

The server script shown in Figure 3 first stops and removes any currently running model containers. It then creates a new container from the *tensorflow-serving-api* image, and maps ports so that the model can receive *gRPC* requests. Once the container is up, it is ready to receive inference requests from applications that need to use it for making predictions.

The client script shown in Figure 4 takes three examples from a file containing these in CSV format, and

```
(taxi_pipeline) mkd@mkd-ubuntu-20:~$ bash $TFX_EXAMPLES/serving/start_model_server_local.sh $TAXI_DIR/serving_model/chicago_taxi_simple
Pulling the Docker image: tensorflow/serving
Using default tag: latest
latest: Pulling from tensorflow/serving
Digest: sha256:a94b7e3b0e825350675e83b0c2f2fc28f34be358c34e4126a1d828de899ec44f
Status: Image is up to date for tensorflow/serving:latest
docker.io/tensorflow/serving:latest
Starting the Model Server to serve from: /home/mkd/taxi/serving_model/chicago_taxi_simple
Model directory: /home/mkd/taxi/serving_model/chicago_taxi_simple
Error response from daemon: No such container: chicago_container
Error: No such container: chicago_container
50eb325f9f5954fd622c33a46da52dbf0bb9bd7d4ce24cf6f2db9a08b9238985
```

Figure 5: Deploying the model manually